ENSEMBLE METHODS · MODULE 2

Beyond GBDT: The *Modern Boosting* Family

A complete, example-driven guide to XGBoost, LightGBM, Stacking and Cascading — what each technique actually optimizes, why it works, and where it earns its place in production.

5 core techniques | Recap → Cascading | math · code · real-world cases

WHAT'S INSIDE

01 From GBDT to Stochastic GBDT — the foundation

02 XGBoost — Extreme Gradient Boosting — speed by engineering

03 LightGBM — GOSS & EFB — faster still

04 Stacking — the Kaggle weapon

05 Cascading — when errors are expensive

06 Decision guide & cheat-sheet — when to use what

CHAPTER 01

From GBDT to Stochastic GBDT

Everything in this guide is a variation on one idea you already know: **gradient-boosted decision trees**. So let's anchor on that first, then watch how each new technique either makes it *faster*, *more accurate*, or *safer to deploy*.

The one-line recap of GBDT

Standard GBDT builds trees *sequentially*. Each new tree is trained to predict the **pseudo-residuals** (the part of the target the previous trees got wrong), and predictions are **additively combined**:

$$\hat{y}(x) \;=\; h_0(x) \;+\; \sum_{j=1}^{M} \alpha_j \, h_j(x)$$

Here h_0 is the initial guess, each h_j is a small tree (a "weak learner"), α_j is its learning-rate-scaled contribution, and M is the total number of trees. The catch: because every tree chases the previous tree's mistakes, plain GBDT **overfits easily** and is **slow to train**.

Stochastic GBDT — add randomness on purpose

Stochastic GBDT is simply:

> DEFINITION
>
> **Stochastic GBDT = GBDT + randomization**, where randomization comes from **Row Sampling (R.S.)** and **Column Sampling (C.S.)** — each tree sees only a random subset of rows and a random subset of features.

Why does this help? The same reason it helps in Random Forest: injecting randomness **reduces variance**. If every tree trained on the exact same data and features, they'd all make correlated mistakes. Forcing each tree to look at a different slice de-correlates them, so their combined prediction generalizes better.

REAL-WORLD INTUITION

Think of diagnosing a tricky patient by polling 50 doctors. If you give all 50 the *identical* chart, they'll likely repeat the same oversight. If instead each doctor sees a slightly different subset of the labs and symptoms, their blind spots don't line up — and the *panel's* consensus is far more reliable. Row/column sampling is that "different subset per doctor" trick.

How do you actually get Stochastic GBDT?

- **scikit-learn:** use `GradientBoostingClassifier` with the `subsample` parameter (row sampling, e.g. `0.8`) and `max_features` (column sampling).
- **The better implementation** lives in the **XGBoost library** — same idea, dramatically faster engineering. That's our next chapter.

> WORTH KNOWING
>
> People sometimes call `XGBClassifier` "sklearn's GBDT, but better." That's a fair mental model: it follows the scikit-learn API so it feels familiar, but underneath it's a separate, heavily optimized C++ engine. *(Aside: scikit-learn itself is an open-source project — originally out of INRIA — not a product owned by Google. It's community-maintained.)*



"Extreme" = extreme in training speed — more stochasticity, smarter use of compute.

The lineage. *Each step inherits the previous one and adds something. XGBoost/LightGBM don't change the math much — they change the engineering.*

CHAPTER 02

XGBoost — Extreme Gradient Boosting

XGBoost was published as a research project in **2016** (Chen & Guestrin, KDD). The whole point of the name "extreme" is performance: it does the *same* gradient boosting, but with a stack of engineering optimizations that make it train far faster and overfit less.

> THE SHAPE OF IT
>
> Originally written in **C++** for speed. To make it usable by data scientists, the authors wrapped it in the **scikit-learn interface** — that's the `XGBClassifier` / `XGBRegressor` Python API you call. Install with `pip install xgboost`.

The three optimizations that make XGBoost fast

These aren't dramatic — the authors describe them as "not very fancy, slightly better." But stacked together they matter enormously. The key insight behind all of them: **find work that is independent, and do it in parallel.**

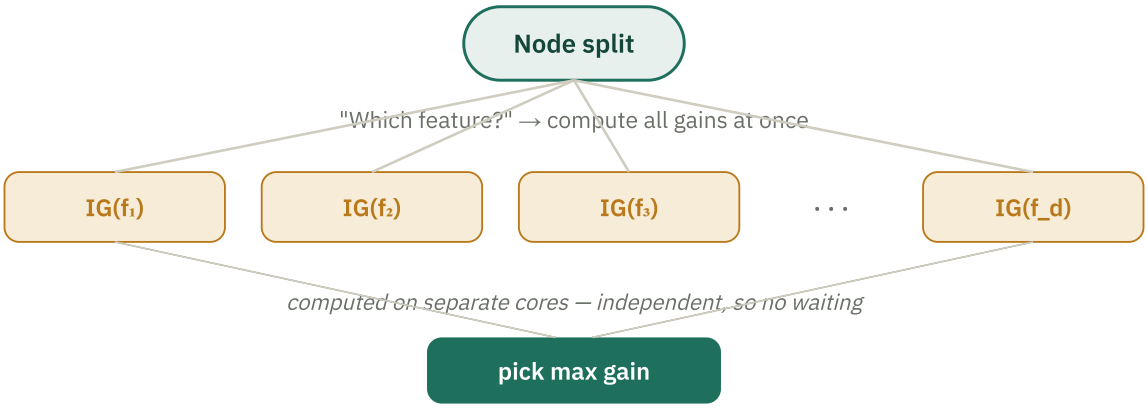
① Parallelizing feature selection at a node

To split a node in a decision tree, you must answer: *which feature gives the best split?* With features $f_1, f_2, \ldots, f_d$, you compute an Information Gain for each:

$$IG(f_1),\ IG(f_2),\ \ldots,\ IG(f_d) \quad \Rightarrow \quad \text{pick the max}$$

Crucially, $IG(f_1)$ does **not depend on** $IG(f_2)$ — these computations are **independent**. So XGBoost computes them **in parallel** across CPU cores (the `n_jobs` parameter), then picks the winner.

> **IMPORTANT CAVEAT**
>
> This is *not* parallelizing across trees. Boosting trees are **sequential by nature** — tree $j+1$ needs tree j's residuals, so you can't train all trees at once. The parallelism happens *inside* a single tree, at the moment of choosing a feature.



Feature-level parallelism. *The gains are mutually independent, so they're evaluated simultaneously rather than one-by-one.*

② Parallelizing the subtrees

Once a node picks its best feature (say f_2), the data splits into a **left subtree** and a **right subtree**. Those two subtrees are **independent of each other** — building one tells you nothing about the other — so XGBoost can grow them in parallel too.

> REALITY CHECK
>
> Even with both kinds of parallelism, XGBoost will **never be as parallel as Bagging** (Random Forest), where *all* trees are fully independent and train at the same time. Boosting pays an irreducible sequential cost. XGBoost just squeezes parallelism out of every place where it legitimately exists.

③ Smarter thresholds for numerical features (histogram binning)

Here's the classic bottleneck. To split on a numerical feature, a naive decision tree **sorts every value**, then tests each candidate threshold to see which gives the best gain. On large data, sorting + scanning every unique value is **extremely time-consuming** — it's the single biggest reason plain GBDT is slow.

XGBoost's fix: don't test every possible value. Instead, bucket the numerical feature into a fixed number of **discrete bins** (a quantile sketch) and only consider bin boundaries as candidate splits. This is controlled by `max_bin`.

> THE TRADE-OFF
>
> A **larger** `max_bin` → finer thresholds → **better accuracy** but **slower**. A smaller `max_bin` → faster but coarser. This is the knob you tune when speed and accuracy fight each other.

> REAL-WORLD EXAMPLE
>
> Imagine splitting customers on `annual_income` with 1,000,000 rows and 800,000 distinct values. Testing all 800k thresholds is brutal. Binning income into, say, 256 buckets (₹0–2L, 2L–4L, ...) collapses the search to 256 candidates — you lose almost nothing in predictive quality but train an order of magnitude faster.

Regularization in XGBoost

A major reason XGBoost overfits less than vanilla GBDT is built-in regularization, borrowed straight from **linear / logistic regression** intuition.

In linear regression you have weights and you penalize them:

$$\hat{y} = w_0 + \sum_{j=1}^{d} w_j x_j \quad \xrightarrow{\text{regularize}} \quad \text{shrink the } w_j$$

Boosting has an identical shape — compare it with the GBDT equation from Chapter 1. The tree contributions α_j play the role of weights. XGBoost **treats α_j like weights and regularizes them**:

- `reg_alpha` → **L1** penalty on the weights (can drive some to zero — sparsity).
- `reg_lambda` → **L2** penalty on the weights (shrinks them smoothly).

Both can be applied together. This is the structural difference that lets XGBoost stay well-behaved where plain GBDT runs away.

Key hyperparameters

PARAMETER	CONTROLS	NOTE
`n_estimators`	Number of boosting trees	More trees = more capacity (and overfit risk)
`max_depth`	Depth of each tree	Primary overfitting knob
`learning_rate`	Shrinkage per tree (α)	Lower = slower, often better generalization
`subsample`	Row sampling fraction	The "stochastic" in stochastic GBDT

PARAMETER	CONTROLS	NOTE
`colsample_bytree`	Column (feature) sampling	Decorrelates trees
`max_bin`	Bins for numerical splits	Speed ↔ accuracy trade-off
`reg_alpha` / `reg_lambda`	L1 / L2 on weights	Overfitting control
`min_child_weight`	Min instances per leaf	Overfitting control
`objective`	The loss function	e.g. `multi:softmax` for multiclass
`n_jobs`	Parallel cores	Powers optimizations ① and ②

> **MULTI:SOFTMAX EXPLAINED**
>
> `multi:softmax` is the **generalized version of the sigmoid**. Sigmoid gives "probability of one class" (binary); softmax gives "probability across C classes" that sum to 1. Set it with `num_class` for multiclass problems.

Code walkthrough

Hyperparameter-tuning XGBoost with randomized search on a 20-class problem:

```
from xgboost import XGBClassifier
from sklearn.model_selection import RandomizedSearchCV
import datetime as dt

params = {
    "n_estimators":     [50, 100, 150, 200],
    "max_depth":        [3, 4, 5, 7],
    "learning_rate":    [0.1, 0.2, 0.3],
    "subsample":        [0.6, 0.8, 1.0],   # row sampling
    "colsample_bytree": [0.6, 0.8, 1.0],   # column sampling
}
```

```
xgb = XGBClassifier(objective='multi:softmax', num_class=20)

search = RandomizedSearchCV(xgb, param_distributions=params,
                            n_iter=10, scoring='accuracy',
                            n_jobs=-1, cv=3, verbose=2)

start = dt.datetime.now()
search.fit(X_train, y_train)
end = dt.datetime.now()

best = search.best_estimator_
best.fit(X_train, y_train)
print("Model acc", best.score(X_test, y_test))
>> Time taken for fits : 0:09:44   (~10 minutes)

>> Model acc 0.9807
```

THE PUNCHLINE

The same search took **~10 minutes in XGBoost vs > 30 minutes in scikit-learn's GBDT**, for ~98% accuracy. Same algorithm, far better engineering. Remember this number — LightGBM is about to make it look slow.

Quick check — what does `n_estimators` represent in a tree-based model?

- Number of features · Number of parameters · **Number of ensembled trees** · None

▶ Show answer

Which parameters control overfitting in a tree-based model?

▶ Show answer

CHAPTER 03

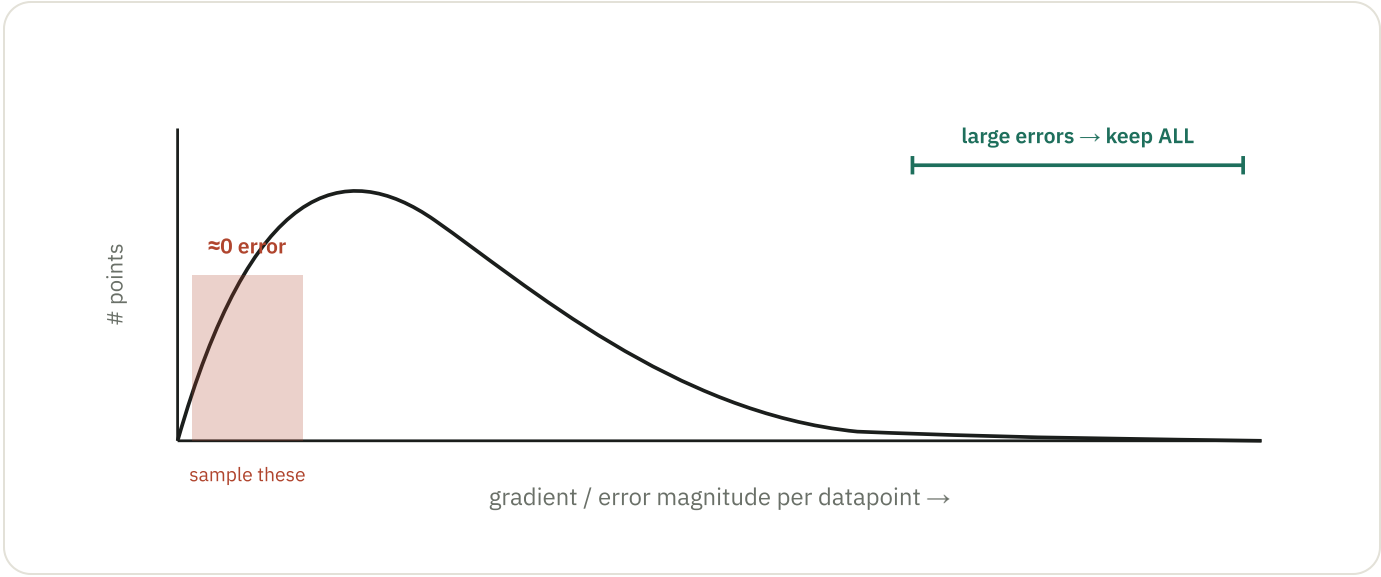
LightGBM — GOSS & EFB

LightGBM (Microsoft, NeurIPS 2017) asks: *XGBoost is fast, but can we go faster without losing accuracy?* Its answer is two clever data-reduction tricks. XGBoost shrinks the *thresholds*; LightGBM shrinks the *data itself* — both the number of **rows** (GOSS) and the number of **features** (EFB).

GOSS — Gradient-based One-Side Sampling

GOSS reduces the **number of datapoints** used to train each tree. The key insight comes from looking at where the model's error lives.

Picture boosting models in sequence: $M_1 \to M_2 \to M_3 \to M_4 \to M_5$. By the time you reach M_3, **a lot of datapoints already have near-zero error** — the model has basically learned them. Only a few thousand "hard" points still carry large errors.



The gradient distribution. *Most points cluster near zero error (the model already knows them). GOSS* **keeps every large-gradient point** *and* **randomly samples** *the easy ones.*

So **"one-side" sampling** means: it samples aggressively on *one side* of the distribution (the easy, low-gradient points) while keeping the informative side intact. Points with tiny

gradients contribute almost nothing to learning — you can drop most of them and barely notice. Points with large gradients are exactly where the next tree should focus, so all of them are retained.

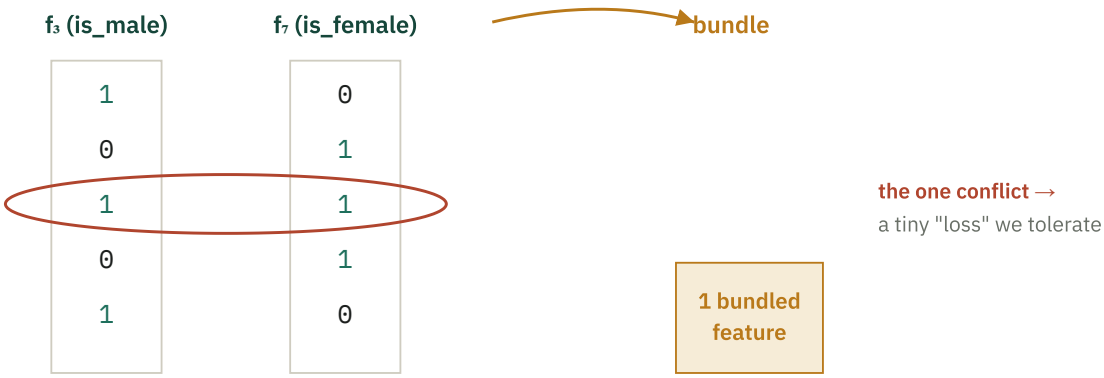
> THE RISK IT MANAGES
>
> Reducing datapoints means the model is **not trained on the full data** — normally a bad thing. GOSS makes it acceptable by being *selective*: it only throws away points that were already learned, never the hard ones. That's the whole trick.

EFB — Exclusive Feature Bundling

While GOSS cuts rows, EFB cuts **columns**. The goal: **reduce the number of features** by *combining* features that are **mutually exclusive** — features that are rarely non-zero at the same time (very common in sparse, one-hot-encoded data).

> WHY NOT JUST USE PCA?
>
> PCA is **lossy compression** — you will always lose ~100% guaranteed some information. EFB is (effectively) **lossless** for truly exclusive features: if two columns are never "1" together, you can pack them into one column without losing which feature fired.

What does "exclusive" mean? When one feature takes value 1, the other (almost) never does. Classic example — a one-hot gender encoding:



Exclusive Feature Bundling. *`is_male` and `is_female` are almost never both 1 — so they collapse into a single feature. The rare conflict is the only information you give up.*

Of course, real datasets have hundreds of features and you **can't check exclusivity by hand**. There can also be non-obvious exclusive pairs — e.g. "likes football?" and "likes swimming?" might happen to be exclusive in *your* dataset even though they aren't logically opposite.

So EFB uses **graph-based techniques**: build a graph where features are nodes and edges connect features that conflict, then find groups (bundles) of mutually-exclusive features to merge. It's a graph-coloring style problem solved greedily.

REAL-WORLD EXAMPLE

A retail model with one-hot encoded `product_category` (200 columns), `payment_method` (12 columns), and `day_of_week` (7 columns). Within each group, exactly one column is 1 per row — perfectly exclusive. EFB can bundle each group down toward a single feature, shrinking 219 sparse columns to a handful, which is a massive speedup with essentially no accuracy cost.

> BONUS DISTINCTION (NOT IN THE SLIDES, BUT IMPORTANT)
>
> LightGBM also grows trees **leaf-wise** (best-first) rather than XGBoost's default **level-wise** growth. Leaf-wise can reach lower loss with fewer nodes but overfits more on

> small data — which is why on small datasets you constrain it with `num_leaves` and `min_data_in_leaf`.

Code walkthrough

```
import lightgbm as lgb
from sklearn.model_selection import RandomizedSearchCV

grid = {
    'learning_rate':    [0.1, 0.3, 0.5],
    'boosting_type':    ['gbdt'],
    'objective':        ['multiclass'],
    'max_depth':        [5, 6, 7, 8],
    'colsample_bytree': [0.5, 0.7],
    'subsample':        [0.5, 0.7],
}

clf = lgb.LGBMClassifier(num_classes=20)
search = RandomizedSearchCV(clf, grid, cv=3, n_jobs=-1, n_iter=10)
search.fit(X_train, y_train)

best = search.best_estimator_
best.fit(X_train, y_train)
print("Model acc", best.score(X_test, y_test))
>> Time taken for fits : 0:00:57   (57 seconds!)
```

> **THE HEADLINE RESULT**
>
> The identical search that took **~10 min in XGBoost** finished in **57 seconds in LightGBM** — with comparable accuracy. That's the combined payoff of GOSS (fewer rows) + EFB (fewer columns) + leaf-wise growth.

When do you use GOSS vs EFB?

▶ Show answer

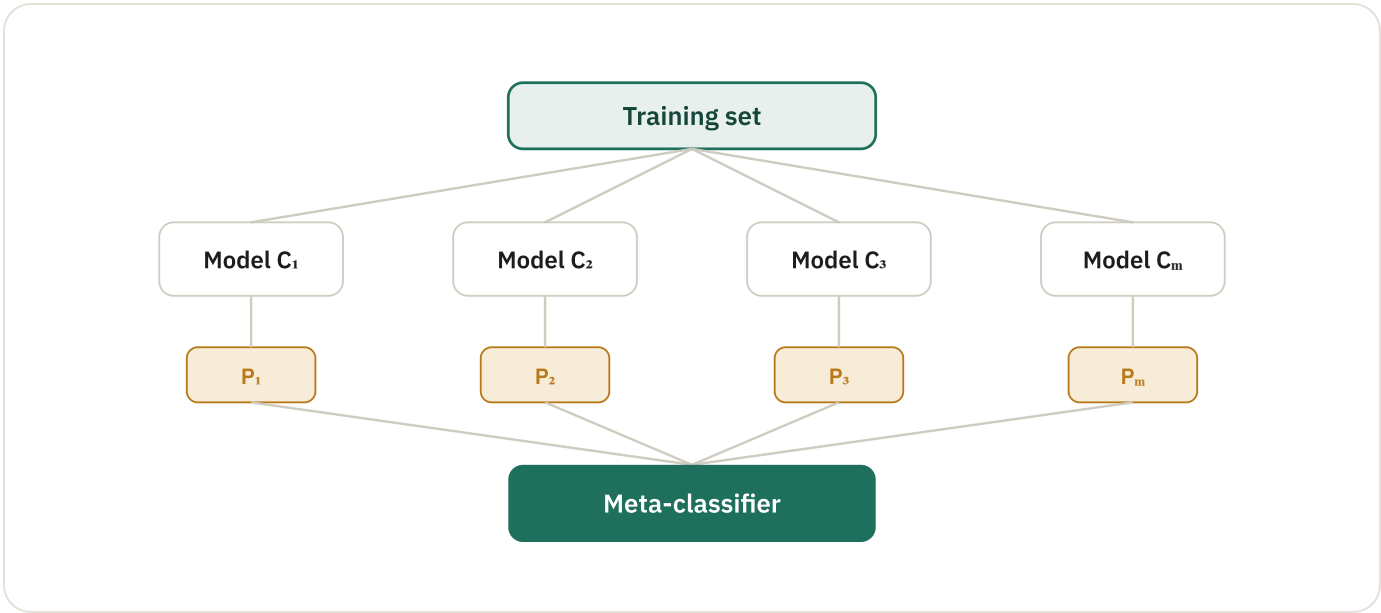
CHAPTER 04

Stacking

We've now exhausted the boosting family. Stacking is a different idea: instead of one model type, **combine many different well-trained models and let another model learn how to combine them.**

How stacking works

1. Train several **diverse, well-tuned base models** $C_1, C_2, \ldots, C_m$ — any algorithms you like (a Random Forest, an SVM, a LightGBM, a logistic regression...).
2. Collect each model's **predictions** on the data.
3. Feed those predictions as *new features* into a final **meta-classifier** (a "meta-learner") that learns the best way to blend them.



Stacking architecture. *Base models' predictions become the meta-learner's input features. The meta-model learns who to trust, when.*

THE KAGGLE STORY

A 4-person Kaggle team splits the work: each member trains a perfectly tuned model of their choice. They then stack all four behind a meta-classifier. Result on the leaderboard:

`single best model … 0.8421` → `stack of all four … **0.8432**`

A gain of ~0.001 — which is meaningless in production but is exactly the margin that decides the top of a competition leaderboard.

So why don't we use it much?

> THE CATCH
>
> Stacking buys you a **tiny** accuracy gain at a **large** cost: you must train, serve, monitor, and maintain *many* models plus a meta-model. The latency, infra, and maintenance burden almost never justify a third-decimal-place improvement. So in practice, **stacking is mostly used to win Kaggle competitions**, not to ship products.

Implementation: see mlxtend's `StackingClassifier` .

What is stacking?

▶ Show answer

CHAPTER 05

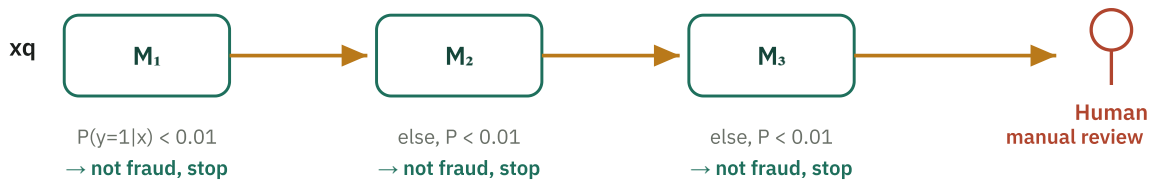
Cascading — chaining of models

Cascading is the most **industry-relevant** technique here. The idea: **chain models one after another**, where each model confidently handles the easy cases and passes only the uncertain ones down the chain — with a **human as the final safety net**.

> WHEN TO REACH FOR IT
>
> Cascading is used when the **penalty of making an error is huge**. If a wrong prediction is catastrophic (financial loss, a missed cancer), you don't let a single model decide alone — you build a chain that only commits when it's extremely confident.

Worked example: fraud detection

Target: $y = 1$ means fraud, $y = 0$ means genuine. A query transaction x_q enters the chain:



Each stage only forwards cases it cannot confidently clear.
The human handles the tiny, genuinely ambiguous remainder — then confirms with the customer.

A cascade. *Confident "clears" exit early; ambiguity flows downstream until a model — or finally a person — resolves it.*

Walking the chain:

- **M_1:** if $P(y = 1 \mid x_q) < 0.01$, declare *not fraud* and stop. This first model's job is mostly to **strip out the obviously-genuine customers**.
- **M_2** trains on what's left and repeats the test on the harder remainder.
- **M_3** does the same on the still-harder remainder.
- If *even after all models* we're still unsure, a **human reviewer** steps in — e.g. an automated call: *"Did you make this transaction? Press 1 to confirm or 2 to deny."*

KEY STRUCTURAL FACT

Every model in the cascade is trained on a **different dataset** — the data the previous models couldn't resolve ($D - D_n$). Each stage specializes in progressively harder cases. This is what distinguishes cascading from stacking, where all models see the same data.

WHERE IT'S USED IN INDUSTRY

Cancer detection: a cheap screening model clears clearly-healthy scans; suspicious ones escalate to a more sensitive model; the truly ambiguous go to a radiologist. **Finance / fraud:** exactly the chain above. **Content moderation & spam:** fast filters auto-clear the obvious, edge cases route to heavier models, then to human moderators. In all of them, the cost of a false negative is too high to trust one model.

Why is cascading helpful?

▶ Show answer

CHAPTER 06

Decision guide & cheat-sheet

What is each technique really doing?

TECHNIQUE	CORE MOVE	WINS YOU	REACH FOR IT WHEN…
Stochastic GBDT	GBDT + row/col sampling	Lower variance	Plain GBDT is overfitting

TECHNIQUE	CORE MOVE	WINS YOU	REACH FOR IT WHEN…
XGBoost	Parallel splits + binning + L1/L2 reg	Speed + robustness	Strong tabular baseline; default first choice
LightGBM	GOSS (rows) + EFB (cols) + leaf-wise	Much more speed	Large / wide / sparse data; need fast iteration
Stacking	Meta-model over diverse models	Last 0.1% accuracy	Competitions; accuracy is everything, cost ignored
Cascading	Chain models + human fallback	Controlled error cost	Misclassification is very expensive

XGBoost vs LightGBM at a glance

	XGBOOST	LIGHTGBM
Speed strategy	Bin thresholds (`max_bin`)	Drop easy rows (GOSS) + bundle cols (EFB)
Tree growth	Level-wise (default)	Leaf-wise (best-first)
Typical speed	Fast	Faster (often 5–10×)
Small data	More forgiving	Can overfit — constrain `num_leaves`
Published	2016 (KDD)	2017 (NeurIPS)

> **THE MENTAL MODEL TO KEEP**
>
> Boosting's accuracy story barely changes across XGBoost and LightGBM — what changes is **how cleverly each one avoids unnecessary work**. XGBoost parallelizes and bins; LightGBM throws away data it doesn't need. Stacking and cascading aren't about speed at all — they're about *combining* models for accuracy (stacking) or for *safety* (cascading).

ONE-PARAGRAPH SUMMARY

Start from sequential GBDT. Make it less overfit-prone with **row/column randomness** (Stochastic GBDT). Make it *fast* with engineering — parallel split-finding and histogram binning (**XGBoost**), or by shrinking the data through gradient-based sampling and feature bundling (**LightGBM**). When you need the absolute last fraction of accuracy and don't care about cost, blend diverse models with a meta-learner (**Stacking**). And when a wrong answer is too costly to risk, chain models with a human backstop (**Cascading**).

Compiled from lecture notes on *Other Ensemble Techniques* · References: XGBoost paper (2016) · LightGBM paper (2017) · XGBoost docs · LightGBM docs · mlxtend stacking